

Intro to Regular Expressions

 <https://github.com/learn-co-curriculum/python-p3-intro-to-regex>  <https://github.com/learn-co-curriculum/python-p3-intro-to-regex/issues/new>

Learning Goals

- Validate that strings match specific patterns using regular expressions.
- Search for strings that match specific patterns using regular expressions.

Key Vocab

- **Regular Expression:** a sequence of characters used to search for a pattern inside of a string.
- **Pattern:** a description of sequences of characters that share certain traits with one another. Sequences do not need to be the same length or share any common characters to pattern match. Also called a **filter**.

Introduction

Say you're working at your new job as a developer and your supervisor asks you to build in validation for the email field in the company's signup form. There have recently been a lot of sign-ups with invalid email addresses (e.g., "joeflatiron.com", "@helloworld.com", and "\$%!_adam@gmail.com (mailto:-adam@gmail.com)"). First, you sit down and come up with a set of rules that any email address should adhere to (stop reading and see how many you can come up with):

- Numbers, letters, dashes, and underscores are ok.
- Uppercase and lowercase letters are ok.
- **% , ? , \$, ! , *** are not valid characters.
- There must be an **@** separating the local part from the domain part.

- There must be at least one period in the domain part (e.g., gmail.com).
- Two periods in a row (..) are not allowed.
- The local (first) part of the email cannot start with a period.

We now have a **pattern** that we know all email addresses must follow. We use **regular expressions** (usually shortened to **regex**) to encode these patterns for matching, searching, and substitution. Here's a sample regular expression for email validation:

```
r'\b[A-Za-z0-9._%+-]+\@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
```

If this doesn't make any sense, don't worry. We'll be covering how to write and read regular expressions shortly.

(There are actually a [LOT more rules](https://en.wikipedia.org/wiki/Email_address#Domain_part) [↗](https://en.wikipedia.org/wiki/Email_address#Domain_part) https://en.wikipedia.org/wiki/Email_address#Domain_part for email addresses, but you get the point.)

History

RegEx came about in the 1950's and 1960's in various forms. Among the first appearances of regular expressions in program form was when Ken Thompson built [Stephen Cole Kleene](https://en.wikipedia.org/wiki/Stephen_Cole_Kleene) [↗](https://en.wikipedia.org/wiki/Stephen_Cole_Kleene) https://en.wikipedia.org/wiki/Stephen_Cole_Kleene's notation into the editor QED as a means to match patterns in text files. Since then, there have been various implementations of regular expressions developed.

We'll be using Python regular expressions, an implementation mostly based off the PERL language. A key difference between the two is that regex in Python requires us to import the `re` module, where regex is natively supported in PERL and many other languages.

When to use RegEx

Regular expressions are an extremely powerful way to search through strings and blocks of text for specific patterns. They can be used for data validation, searching, mass file renaming, and finding records in a database. Use them carefully. They are like a surgeon's scalpel: able to do a lot of harm or good, depending on how skillfully they are wielded.

Resources

- [Python 3 Documentation](https://docs.python.org/3/) [↗](https://docs.python.org/3/) <https://docs.python.org/3/>

- [re - Regular expression operations - Python](https://docs.python.org/3/library/re.html)  [\(https://docs.python.org/3/library/re.html\)](https://docs.python.org/3/library/re.html)
- [regex101](https://regex101.com/)  [\(https://regex101.com/\)](https://regex101.com/)
- [Python Regular Expressions - Google for Education](https://developers.google.com/edu/python/regular-expressions)  [\(https://developers.google.com/edu/python/regular-expressions\)](https://developers.google.com/edu/python/regular-expressions)